Access Modifiers in Java

16 March 2026 20:31

Access modifiers in Java are keywords that define where a class, method or variable can be accessed. These modifiers help to protect data and control access to code.

What are access modifiers in Java?
Access modifiers are keywords that define the visibility or accessibility of classes, methods, constructors, and variables. They control where these members can be accessed from, such as inside the same class, within the same package, or from other packages. By applying the access modifiers we can restrict or allow access.

There are four types of access modifiers in Java:
1. public
2. private
3. protected
4. default

1. **public Access Modifier:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package, and outside the package. It has the widest scope among all other modifiers.

```
package AccessModifiers;


/**
 * Write a description of class A here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class A
{
    public void msg(){
        System.out.print("\nHello!");
    }

    void display(){ //default access specifier
        System.out.print("\nHello! from display");
    }
}
```

It is accessible only in the same package.

```
public class B
{
    public static void main(String[] args){
        A obj = new A();
        obj.msg();
        obj.display();
    }
}
```

display() is not public in AccessModifiers.A; cannot be accessed from outside package

2. **private Access Modifier:** The excess level of a private modifier is only within the class. It cannot be accessed from outside the class.

```
package AccessModifiers;
```

```java
/**
 * Write a description of class A here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class A
{
    private int x;
    public void msg(){
        System.out.print("\nHello!");
    }

    void display(){ //default access specifier
        System.out.print("\nHello! from display");
    }

    public int getX(){
        return x;
    }

    public void setX(int x){
        this.x = x;
    }
}
```

```java
10 public class Demo
11 {
12     public static void main(String[] args){
13         A ob = new A();
14         ob.x = 10;
15     }
16 }
```

x has private access in AccessModifiers.A

```java
package AccessModifiers;


/**
 * Write a description of class Demo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Demo
{
    public static void main(String[] args){
        A ob = new A();
        //ob.x = 10;
        ob.setX(10);
        System.out.print("\nx = " + ob.getX());
    }
}
```

we need to create setter and getter method for each instance variable, when the access modifier is private.

```java
package AccessModifiers;
```

```java
/**
 * Write a description of class A here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class A
{
    private int x;

    private A(){} //private constructor
    public void msg(){
        System.out.print("\nHello!");
    }

    void display(){ //default access specifier
        System.out.print("\nHello! from display");
    }

    public int getX(){
        return x;
    }

    public void setX(int x){
        this.x = x;
    }
}
```

← Note: A class cannot have private or protected constructor. Except nested class.

```java
10 public class Demo
11 {
12     public static void main(String[] args){
13         A ob = new A();
14         //ob.x = 10;
15         ob.setX(10);
16         System.out.print("\nx = " + ob.getX());
17     }
18 }
```

A() has private access in AccessModifiers.A

```java
10 private class A
11 {
12     priv
13
14     private A(){} //private constructor
15     public void msg(){
16         System.out.print("\nHello!");
17     }
18
19     void display(){ //default access specifier
20         System.out.print("\nHello! from display");
21     }
22
23     public int getX(){
24         return x;
25     }
26
27     public void setX(int x){
28         this.x = x;
29     }
30 }
```

modifier private not allowed here

Error(s) found in class.
Press Ctrl+K or click link on right to go to next error.

Exception: we can design private nested class and constructor, because there scope is within the outer class.

```
package AccessModifiers;


/**
 * Write a description of class A here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class A
{
    private class nestedA{
        private nestedA(){

        }
    }

    private int x;

    private A(){} //private constructor
    public void msg(){
        System.out.print("\nHello!");
    }

    void display(){ //default access specifier
        System.out.print("\nHello! from display");
    }

    public int getX(){
        return x;
    }

    public void setX(int x){
        this.x = x;
    }
}
```

3. **protected Access Modifier:** The protected access modifier is accessible within the package and outside the package, but through inheritance only. The protected access modifier can be applied to the data member, method and constructor. It cannot be applied to the class. It provides more accessibility than the default modifier.

```
package AccessModifiers;


/**
 * Write a description of class A here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class A
{
    /*private class nestedA{
        private nestedA(){
```

```
        }
    }*/

    private int x;

    //private A(){} //private constructor
    public void msg(){
        System.out.print("\nHello!");
    }

    protected void display(){
        System.out.print("\nHello! from display");
    }

    public int getX(){
        return x;
    }

    public void setX(int x){
        this.x = x;
    }
}

package TestingPrint;

import AccessModifiers.A;
/**
 * Write a description of class B here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class B extends A
{
    public void p(){
        display();
    }
}

Test code:
   B ob = new B();
   ob.p();
   ob.msg();

   Output:

      Hello! from display
      Hello!
```

4. **Default Access Modifier:** When we do not specify any specifier, the default access specifier is prefixed by default. The default modifier is accessible within a package only. It cannot be accessed from outside the package. It provides more accessibility than private. But it is more restrictive than protected and public.

```
package AccessModifiers;
```

```
/**
 * Write a description of class M here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
class M
{
    void method1(){
        System.out.print("\nMethod 1");
    }
}
```

default access specified is applied by default.

```
Test code:
   M ob = new M();
   ob.method1();

Output:
   Method 1
```

```
M ob = new M();
    Error: cannot find symbol
     symbol:  class M
     location: class TestingPrint.__SHELL7
```

code written in different package.

not visible in this package.

Access modifiers with method overriding

Modifiers also affect method overriding in inheritance. When a subclass overwrites a method from its parent class, the overridden method cannot have a more restrictive access modifier than the original method.

Rules:

1. A method declared as public in the parent class must be public in the child class.
2. A method declared as protected in the parent class can be protected or public in the child class.
3. A method with default (no modifier) In the parent class can be default or protected or public in the child class but not private.
4. A method declared as private in the parent class is not inherited, so it cannot be overridden.

Accessibility or scope of Java access modifiers

The following table shows the accessibility or scope of Java access modifiers.

Access Modifier	Within Class	Within Package	Outside Package by subclass only (Inheritance)	Outside package
private	Y	N	N	N
default	Y	Y	N	N
protected	Y	Y	Y	N
public	Y	Y	Y	Y